

What is Broken Authentication? ⚡

- Weak or flawed login mechanisms that attackers can exploit.
- Happens when authentication (login) is not implemented securely.
- Allows attackers to guess, steal, or bypass usernames/passwords.
- Includes bad password policies, no rate-limiting, weak session handling.
- Directly linked with identity theft → attacker acts as the victim user.

What is Session Hijacking?

- When an attacker steals or manipulates a user's session ID.
- Session ID = temporary identity of a logged-in user.
- If attacker gets the session ID → attacker becomes that user instantly.
- No need to know username/password → full account control.

How Hackers Exploit Broken Authentication

- Brute-force login attempts.
- Credential stuffing
- Using weak default passwords (admin/admin, test/test).
- No 2FA → easy to bypass.
- Login pages without rate limiting → unlimited password attempts.
- Exploiting weak password reset flows
- Exploiting exposed API endpoints that allow login bypass.

How Hackers Perform Session Hijacking

- Stealing session cookies using:
- XSS attack (steal document.cookie).
- MITM attack on HTTP (non-HTTPS sites).
- Session fixation (attacker sets a session ID → victim logs in using it).
- Malicious browser extensions reading cookies.
- Unsecured WiFi sniffing (public WiFi hijack).
- Reusing stolen cookie in browser → attacker becomes the victim.

Real Impact of Broken Authentication & Session Hijacking

- Full account takeover.
- Unauthorized money transfers.
- Change of email/password (victim completely locked out).
- Access to personal data, messages, company dashboards, admin panels.
- Privilege escalation → from normal user → admin account.
- Identity theft & fraud.
- Complete system compromise.

How to Prevent Broken Authentication

- Strong password policy (min length + complexity).
- Enforce rate limiting (block after 5–10 failed logins).
- Implement Multi-Factor Authentication (MFA/2FA).
- Use secure password hashing (bcrypt, Argon2).
- Disable default passwords.
- Use account lockout & CAPTCHA on multiple failed attempts.
- Secure password reset flow.

How to Prevent Session Hijacking

- Use HTTPS everywhere (no plain HTTP).
- Set secure cookie flags:
 - HttpOnly (JS cannot read cookie → prevents XSS stealing)
 - Secure (sent only over HTTPS)
 - SameSite (prevents CSRF cookie grab)
 - Regenerate session ID after login.
 - Auto logout (session expiration) after inactivity.
 - Don't put session IDs in URL.
 - Use strong, non-predictable session IDs.
- Implement Content Security Policy (CSP) to reduce XSS.

Important Interview Questions

- Difference between Authentication vs Session Management
- Why cookies are the most important target for attackers
- How “Remember Me” features can be risky
- Why HTTP websites are completely vulnerable to session hijacking
- How session fixation works